

Cape Peninsula
University of Technology

FACULTY OF INFORMATICS & DESIGN

Diploma in Information & Communication Technology

PROJECT PRESENTATION 3: PRP372S

Student Name:	**Dumisane Madondo**
Student Number:	**230949703**
Submission:	**Assessment 2**
Date:	**27October 2026**

Agile Project Management

Project: CAPACITI Candidate Operations and Attendance Platform (Work Integrated Learning, CAPACITI)
My role: Lead Backend Developer
Delivery date: Originally 6 November 2026, brought forward to 29 October 2026

An internal platform that manages daily attendance for CAPACITI candidates in On-Site and Work-From-Home models, with timed check-ins, location security, an AI quality gate and three roles: candidate, tech champion and admin.

1. Agile Team Charter

Element	Content
Purpose	Replace manual attendance tracking with a role-based platform for candidates, tech champions and admins
Product Owner	CAPACITI
Scrum Master	Omphile Rantswele (Mentor)
Development Team	Dumisane Madondo, Anita Ndabangaye, Siphokazi Lekena, Abram Mohlamonyane, Cherish Nwachukwu
Values	Accountability for own slice, transparency about blockers, protecting candidate data, backend as the single source of truth
Definition of Done	Slice implemented on its feature branch, compiles, merged into develop through a reviewed pull request, aligned with the agreed API contract
Boundaries	No secrets in Git. Members edit only files they own. The frontend uses backend field names, and features the backend lacks are marked mock-only.
Communication	Teams messages, morning stand-ups, Bruno collection for API checks
Key Dates	Internal deadline 27 Oct, buffer 28 Oct, final delivery 29 Oct 2026

Reflection: Agile Team Charter (STAR Method)

Situation: I joined a five-member CAPACITI team to build the attendance platform, and the delivery date moved from 6 November to 29 October, leaving about eight days to build.

Task: As lead backend developer, I needed working agreements that let five people build in parallel without conflicts.

Action: On 19 and 20 October I put in place ownership by file, per-member feature branches, pull request reviews by me, contracts agreed before coding, secrets in environment variables, and a daily morning stand-up with notes taken by Cherish Nwachukwu.

Result: Members built their slices without overwriting each other and develop stayed stable. However, a Supabase secret still reached a commit and was blocked by GitHub push protection.

Reflection: The concrete agreements, such as who owns which file and which branch to use, worked best. The secret incident showed that "secrets in environment variables" had to be enforced from day one rather than assumed. Next time I will write the charter before any code and add a rule on pinning framework versions, since Spring Boot 4.1.1 costs us time. The charter links to the other four topics: its working agreements created the stand-ups in the logbook, and its ownership rules decided how stories were split into tasks.

2. User Stories and Tasks

ID	User Story	Tasks	Owner	Status
US1	As a candidate, I want to clock in and out so that my daily attendance is recorded	Attendance entity and endpoints; status engine (arrival and departure stored separately); on-site check against campus IP whitelist	Anita Ndabangaye	Done
US2	As a candidate, I want my location verified so that on-site attendance is genuine	Geofence and location verification; IP whitelist; trusted proxy handling	Anita Ndabangaye	Done (proxy setting to do)
US3	As a candidate, I want to answer three timed check-ins so that I show I am working	Window calculator; prompt generator; quality attempts; Gemini client and AI quality gate	Siphokazi Lekena	Done

US4	As a candidate, I want to apply for WFH so that I can work remotely when approved	WFH request entity and endpoints; approval flow	Abram Mohlamonyane	Done
US5	As a candidate, I want to log blockers and IT issues so that they get resolved	Support ticket entity and endpoints; blocker queue; prolonged-ticket scheduler	Abram Mohlamonyane	Done
US6	As a candidate, I want to view my scorecards so that I can see my progress	Assessment and ScorecardEntry entities; endpoints; candidate UI; database script 12	Anita Ndabangaye, Cherish Nwachukwu	Done (script 12 in progress)
US7	As a tech champion, I want to monitor attendance and approve or reject WFH so that I can manage my group	Dashboards; WFH approval endpoints; excuse lateness; resolve tickets	Cherish Nwachukwu, Abram Mohlamonyane	Done
US8	As a tech champion, I want to score candidates so that I can give feedback	Mentor Scorecards backend and UI; group defined by users.champion_id	Anita Ndabangaye	Done
US9	As an admin, I want to manage users, the IP whitelist and the audit log so that the system stays secure	User management; IP whitelist; audit log; reports	Dumisane Madondo, Anita Ndabangaye, Cherish Nwachukwu	Done
US10	As an admin, I want automated reports and Teams alerts so that I do not chase data manually	Attendance and prompt schedulers; reports; Teams webhook client; signed file links	Cherish Nwachukwu	Done (live Teams test to do)
US11	As any user, I want role-based login so that I	Users and login; JWT service and filter; BCrypt; role-	Dumisane Madondo	Done

	only see what my role allows	based security config; JSON 401/403 handlers		

Reflection: User Stories and Tasks (STAR Method)

Situation: The team built a frontend-first prototype without Figma wireframes, and the screens included cohort, stream, candidate ID, notifications and break buttons that the backend did not have.

Task: Define the user stories as the real source of truth for what the platform should do.

Action: I turned the needs of the three roles into eleven stories and split them into backend slices owned by one member each, using the backend naming as the contract. For example, US3 (timed check-ins) became the window calculator, prompt generator, and Gemini quality gate for M3.

Result: Every member had specific, testable tasks, and the slices were implemented. But the stories had no formal acceptance criteria, so the User entity did not match the users table and the mismatch only surfaced during integration.

Reflection: Writing stories after the prototype cost us rework. Next time I would write the stories first and base the database scripts on them, with acceptance criteria for each. The stories feed directly into the product backlog, where each became a prioritised item.

3. Product Backlog

Priority	Item	Type	Owner	Status
1	Database schema, RLS and storage policies in Supabase	Feature	Lead	Done
2	Role-based login with JWT and user management	Feature	M1	Done
3	Attendance clock-in/out and status engine	Feature	M2	Done
4	Location security (IP whitelist and geofence)	Feature	M2	Done
5	Timed check-ins with AI quality gate	Feature	M3	Done

6	WFH requests and approval	Feature	M4	Done
7	Support tickets, blockers and prolonged scheduler	Feature	M4	Done
9	Frontend for all three roles	Feature	Frontend team	Done (rebuilt from AI Studio design)
10	Mentor Scorecards	Feature	M2	Done
11	Fix Supabase smoke test (missing PostgreSQL driver)	Bug	Lead	In progress
12	Rotate exposed secrets and complete GitHub push	Bug	Lead	In progress
13	Wire every frontend screen to real endpoints	Improvement	Frontend team	In progress
14	Run scorecards script 12 and create storage buckets	Improvement	M5	In progress
15	Trust X-Forwarded-For only from a trusted proxy	Improvement	Lead	To do
16	End-to-end tests for all three roles; live Gemini and Teams tests	Testing	Team	To do
17	Update presentation script to match the build	Improvement	Team	To do
18	Docker and deployment	Improvement	Lead	To do

Reflection: Product Backlog (STAR Method)

Situation: With the delivery date moved to 29 October, the team had very little time and could not build everything.

Task: Decide what to build first and keep scope under control.

Action: I prioritised by dependency, so the database and authentication came first and the AI gate and schedulers later. I accepted Mentor Scorecards as a late extra only after the core flows were complete, and I marked frontend features the backend lacked as mock-only instead of adding them to the backlog.

Result: All core features were implemented and scope stayed controlled. Bugs such as the smoke test and the secret rotation entered the backlog late, after they had already blocked us, and script 12 for the scorecards remained unconfirmed.

Reflection: Prioritising by dependency worked well under time pressure, but adding a late feature carried a real risk. Next time I would log technical risks as backlog items from day one. The backlog feeds the sprint backlog, where the top items were pulled into a sprint.

4. Sprint Backlog

Sprint 3: Backend Feature Slices (20 to 25 Oct)

Sprint Goal: Each member delivers their slice end to end on the agreed contracts.

Item	Tasks	Owner	Status
Users, login and auth	User management; JWT login; audit log	Dumisane	Done
Attendance and location	Campus and IP whitelist; attendance; status engine; geofence	Anita	Done
Timed check-ins	Window calculator; prompt generator; Gemini client; quality gate	Siphokazi	Done
WFH and support	WFH requests and approval; tickets; blocker queue; prolonged scheduler	Abram	Done
Dashboards and automation	Dashboards; reports; schedulers; Teams client; signed file links	Cherish	Done
Demo data	Demo data seeder	Dumisane, Anita, Cherish	Done
Live integration tests	Gemini and Teams payload checks	Team	Carried over to Sprint 5

Sprint 5: Extra Feature and Hardening (26 to 29 Oct)
Sprint goal: A tested, demonstrable platform for 29 Oct.

Sprint 5: Extra Feature and Hardening (26 to 29 Oct)

Sprint Goal: A tested, demonstrable platform for 29 Oct.

Item	Status
Mentor Scorecards backend and UI	Done
Script 12 and storage buckets	In progress
Wire frontend to real endpoints	In progress
End-to-end test of all three roles	To do
Update presentation script	To do

Reflection: Sprint Backlog (STAR Method)

Situation: Sprint 3 ran from 20 to 25 October, with five members each owning a backend slice.

Task: Deliver every slice end to end on the agreed contracts.

Action: I agreed contracts and ownership before the sprint started, so all five slices were built in parallel, and I used the morning stand-ups to spot blockers early.

Result: All five slices were implemented. When they came together we hit compile errors from mismatched contracts, such as Role versus UserRole and findByEmail versus findByEmailIgnoreCase, and Spring Boot 4.1.1 caused further problems. This cost about a day of integration fixes, and the live Gemini and Teams tests carried over into Sprint 5.

Reflection: A clear sprint goal and agreed ownership made parallel work possible, but integrating only at the end was a mistake. Next time I would add an integration checkpoint halfway through the sprint. The sprint backlog connects to the logbook, because the daily stand-ups recorded how each task moved.

5. Agile Logbook

Taskboard (as at 26 Oct 2026)

Product Backlog	In Progress	Completed
Sprint 1: Prototype (frontend first)	None	Role-based screens on sample data; 12-slide prototype presentation script; prototype delivered
Sprint 2: Planning and Foundation (19 to 21 Oct)	None	Business rules and shift windows; script vs build gap analysis; backend plan, ownership split and timeline; Spring Boot skeleton and Git branching; exception package, Helper, TimeUtil and ten enums; Copilot base and per-member prompts
Sprint 3: Database and Security (21 to 22 Oct)	Supabase smoke test; storage buckets	8-table schema, RLS and storage policies; security, CORS, JWT filter and handlers; /api/v1 standard and Spring Boot 3.5.16; secrets moved to environment variables
Sprint 4: Backend Feature Slices (20 to 25 Oct)	Live Gemini and Teams testing	Auth and users; attendance, location and status engine; check-ins and AI quality gate; WFH and support tickets; dashboards, reports, schedulers and Teams client; demo data seeder
Sprint 5: Integration and Frontend Alignment (22 to 26 Oct)	Wire frontend to real endpoints; secret rotation and GitHub push	Compile, config and merge fixes; User entity aligned with table; frontend rebuilt from AI Studio design
Sprint 6: Hardening (26 to 29 Oct)	Scorecards script 12	Mentor Scorecards backend and UI

WALKTHROUGH

1. **What do we need to do to**	Confirm the Supabase connection and schema validation, wire every frontend screen to the real endpoints, and run end-to-end

advance this piece of work?	tests for the candidate, tech champion and admin roles. The scorecards database script (script 12) must also be run before Mentor Scorecards can be validated.
2. Is anyone working on anything that is not on the board?	Mentor Scorecards was added late as an extra feature, outside the original plan.
3. What do we need to finish as a team?	Confirm the secrets were rotated and the push to GitHub succeeded, create the storage buckets, update the presentation script to match the final build, and prepare for the internal deadline on 27 Oct (buffer 28 Oct) and the final delivery on 29 Oct.
4. Are there any bottlenecks or blockers to the flow of work?	Resolved: Spring Boot 4.1.1 caused compile errors (pinned to 3.5.16), a JDK older than 21 blocked builds, a duplicate main class and merge conflicts needed fixing, and the User entity did not match the users table. Still open: the Supabase smoke test (missing PostgreSQL driver), GitHub push protection blocking a secret in .env.example, and live testing of the Gemini and Teams integrations.

LOGBOOK

Date:	19 October 2026	
Monday	1. What did I complete since the last standup?	Compared the prototype script against the build and drafted the backend plan, timeline and ownership split. Created the Spring Boot skeleton with the exception package. Prepared the first prompts for Member 1.
	2. What am I planning to complete between now and the next standup?	Write the base Copilot prompt and the prompts for Members 2 to 5.
	3. What are my impediments (or risks or problems)?	The delivery date was brought forward from 6 November to 29 October, which left no time for Figma wireframes. The frontend also showed fields the backend did not have.

Date:	20 October 2026	
Tuesday	1. What did I complete since the last standup?	Finished the base Copilot prompt and the per-member prompts so each member could start their slice.

	2. What am I planning to complete between now and the next standup?	Agree the controller style, then set up the config classes, the location stub and the demo data prompt.
	3. What are my impediments (or risks or problems)?	No blockers

Date:	21 October 2026	
Wednesday	1. What did I complete since the last standup?	Set the controller style, config prompts, location stub and demo data prompt. Decided on the /api/v1 path standard. Pinned Spring Boot to 3.5.
	2. What am I planning to complete between now and the next standup?	Set up the Supabase database, the RLS and storage scripts, and the connection.
	3. What are my impediments (or risks or problems)?	Spring Boot 4.1.1 caused Jackson and WebFlux incompatibilities with our Boot 3 assumptions. Fixed by pinning to 3.5.16.

Date:	22 October 2026	
Thursday	1. What did I complete since the last standup?	Created the 8-table schema, RLS, deny policies and the storage policy in Supabase. Wrote the Member 5 instructions and ran the first smoke test. Fixed compile errors, deleted a duplicate main class and resolved merge conflicts. Aligned the User entity with the table.
	2. What am I planning to complete between now and the next standup?	Prepare the backend summaries and the frontend design prompts.
	3. What are my impediments (or risks or problems)?	The smoke test used the dashboard URL instead of the JDBC URL, then failed on a missing PostgreSQL driver. GitHub push protection blocked a Supabase secret in .env.example, so the key had to be rotated and the commits squashed.

Date:	23 October 2026	
Friday	1. What did I complete since the last standup?	Wrote the backend summaries and the frontend design and polish prompts. Found that the candidate dashboard did not match the backend. Replaced the frontend with the Google AI Studio build, aligned to the backend types, with mock-only features marked.
	2. What am I planning to complete	Plan Mentor Scorecards and prepare progress reports.

	between now and the next standup?	
	3. What are my impediments (or risks or problems)?	The frontend did not follow the design and fonts did not load, which caused the rebuild. Wiring screens to real endpoints is still open.

Date:	26 October 2026	
Monday	1. What did I complete since the last standup?	Planned and delivered Mentor Scorecards (backend and UI, Member 2). Recapped the check-in rules and wrote the progress reports.
	2. What am I planning to complete between now and the next standup?	Confirm the smoke test and the GitHub push, run script 12, and start end-to-end testing for the 27 Oct internal deadline.
	3. What are my impediments (or risks or problems)?	Script 12, the storage buckets and the final GitHub push are all unconfirmed. Live Gemini and Teams testing has not been done.

Reflection: Agile Logbook (STAR Method)

Situation: The team held a stand-up every morning before working on the platform, and Cherish Nwachukwu took notes of each session.

Task: Keep a record of what was completed, planned and blocked, so progress could be traced.

Action: I logged each stand-up from 19 to 26 October using the template's three questions, alongside a taskboard and a walkthrough of where the team stood.

Result: The log shows what went well, such as agreeing contracts and file ownership early and recovering quickly from build errors. It also exposes what did not: the frontend-first approach created rework, I pinned the framework version late, and a secret reached a commit. The moved deadline is visible in the earliest entries and explains the pressure behind many later decisions.

Reflection: Reading the log back, I noticed we often discussed blockers but rarely reviewed how we worked, so a proper retrospective would have helped. Next time I would pin versions on day one, sketch wireframes even without Figma, and hold a short retrospective after each sprint. The logbook ties the other four topics together: the charter set the stand-up routine, and the stories, backlog and sprint are what the entries record.